

オンボーディング: C2担当（hanako）

作成日: 2026-05-23

対象: hanako（C2担当: Channel）

所要時間: 約30分で読める

1. IRCとは

- **Internet Relay Chat** の略。1988年生まれのテキストチャットプロトコル
- 1つのサーバーに複数クライアントが接続し、チャンネル（部屋）で会話する
- Slack/Discordの先祖。今も使われている

2. ft_irc課題の目標

C++98でIRCサーバーを作る。

できるようになること:

- irssi（IRCクライアント）から接続できる
- ユーザー登録（PASS/NICK/USER）ができる
- チャンネルに入って会話できる（JOIN/PRIVMSG）
- オペレーター機能（KICK/INVITE/TOPIC/MODE）が動く

3. 全体アーキテクチャ（4層）

OSI参照モデル準拠: アプリケーション層（抽象度高）が上、ネットワーク層（低レイヤー）が下。



データの流れ（上向き: 受信 / 下向き: 送信）:

【受信】 クライアント → A層(recv) → B層(parse/dispatch) → C1/C2層(状態更新)

【送信】 C1/C2層 → B層(reply生成) → A層(send) → クライアント

4. 君の担当: C2層

担当クラス

クラス	役割
Channel	1つのチャンネルの状態（メンバー、オペレーター、トピック等）
ChannelModes	チャンネルのモード状態（+i, +t, +k, +l）

Channelが持つもの

```
class Channel {
    std::string      _name;        // チャンネル名 (例: "#general")
    std::string      _topic;       // トピック (部屋の説明文)
    std::set<Client*> _members;    // 参加中のユーザー
    std::set<Client*> _operators;  // オペレーター権限を持つユーザー
    std::set<Client*> _invited;    // 招待済みユーザー (+i モード用)
    ChannelModes     _modes;      // モード状態
};
```

ChannelModesが持つもの

```
class ChannelModes {
    bool      _inviteOnly;    // +i: 招待制
    bool      _topicRestricted; // +t: トピック変更をオベ限定
    std::string _key;         // +k: パスワード
    int       _limit;         // +l: 人数制限 (-1 = 無制限)
};
```

各モードの意味

モード	意味	例
+i	招待制。招待されないと入れない	秘密の部屋
+t	トピック変更はオペレーターのみ	荒らし防止
+k	パスワード必須	JOIN #room password
+l	人数制限	最大10人まで
+o	オペレーター権限 (※これは _operators 集合で管理)	管理者

5. オペレーター (operator) の仕組み

重要: operatorはChannelが管理する

```
Channel #foo
├ members: [Alice, Bob, Carol]
└ operators: [Alice]           ← Aliceは#fooでオペレーター

Channel #bar
├ members: [Alice, Bob]
└ operators: [Bob]             ← Aliceは#barではオペレーターではない
```

理由: 1人のユーザーが複数チャンネルで異なる権限を持てるから。

Operator Bootstrap (新規チャンネル作成時)

```
JOIN #newchannel  ← 存在しないチャンネルに入ろうとした
    ↓
チャンネルを新規作成
    ↓
最初のJOIN者が自動的にオペレーターになる
```

これがないと、誰もKICK/INVITE/TOPIC/MODEを使えなくなる。

6. C2が関わるコマンド

コマンド	動作	C2の責務
JOIN #ch	チャンネル参加	member追加、+i/+k/+l チェック
PART #ch	チャンネル退出	member削除
KICK #ch user	強制退出	オベ権限チェック、member削除
INVITE user #ch	招待	オベ権限チェック、invited追加

コマンド	動作	C2の責務
TOPIC #ch :text	トピック設定	+t時のオペ権限チェック
MODE #ch +i	モード変更	オペ権限チェック、モード更新
PRIVMSG #ch :text	チャンネル発言	member一覧取得（配送先特定）

7. 読むべきドキュメント（この順番で）

順序	ファイル	内容	時間目安
1	このファイル	全体像把握	10分
2	reading_guide_common.md	共通概念、データフロー図	10分
3	design.md Section 3.4, 7	Channel/ChannelModesの詳細設計	20分
4	interface.md Section 8	Channel/ChannelModesの関数一覧	15分
5	reading_guide_C2.md	C2向け詳細ガイド	10分

RFC（後で読む）：

- RFC 1459 Section 1.3（チャンネル概念）
- RFC 1459 Section 4.2（JOIN/PART/KICK等）
- RFC 2812 Section 3.2（チャンネルコマンド詳細）

8. 最初の一步

1. irssiを試してみる（IRCクライアント体験）

```
brew install irssi
irssi -c irc.libera.chat -n test_nick
# /join #test      ← チャンネル参加
# /topic #test     ← トピック確認
# /names #test     ← メンバー一覧
# /part #test      ← チャンネル退出
# /quit
```

2. design.md を読む
3. 質問はtorinoueへ

9. よくある疑問

Q: ChannelはClientを所有する？

A: しない。Client* を参照するだけ。Clientの生成・削除はC1層（ServerState）の責務。

Q: Client削除時、Channelはどうなる？

A: ServerState::removeClient() が全Channelから該当Clientの参照を自動削除する。空になったChannelも削除される。

Q: +o（オペレーター）はChannelModesで管理しない？

A: しない。+o は「特定ユーザーへの権限付与」であり、チャンネル自体の属性ではない。Channel::_operators 集合で管理する。

Q: チャンネル名の先頭文字は？

A: # で始まるもののみ対応。&, !, + は課題要件上、対応しない。